

Learning-Augmented Exact Optimization for Asymmetric Pick-and-Place Sequencing

Abstract

We present a learning-augmented exact optimization framework for robotic pick-and-place (PAP) sequencing that combines a graph neural network (GNN) with a constraint programming (CP-SAT) solver. PAP sequencing is an asymmetric travelling salesman problem (ATSP): the travel cost from object i to object j differs from j to i ($c(i, j) \neq c(j, i)$), because the robot’s position after handling object i is fixed by the bin assigned to i ’s type. A GNN trained by imitation learning on integer linear programming (ILP)-optimal prefixes acts as a *search-space reducer* for the exact solver: its logits keep only the top- k outgoing arcs per node, reducing the CP-SAT decision-variable count from $O(N^2)$ to $O(Nk)$. We call the result **LEAP** (Learning-Enhanced Arc Pruning). At $N=200$, LEAP runs $8.75\times$ faster than the unpruned CP-SAT baseline while remaining empirically near-optimal (worst-case gap 0.058% to the unpruned optimum on our test set); because CP-SAT still solves the pruned arc set exactly, this extends practical exact optimization to larger PAP instances. The graph uses three node types (robot, object, and bin), with a distinct bin node per object type, processed by a graph attention network (GAT). The AI contribution is a graph-learning arc-pruning method that makes exact constraint solving tractable on larger instances; the engineering application is travel-time-efficient robotic sorting, with direct relevance to warehouse order fulfilment, automated waste and recycling sortation, and conveyor-based assembly kitting.

Keywords: Learning-Augmented Optimization, Graph Neural Networks, Pick-and-Place Sequencing, Robotic Sorting, Combinatorial Optimization, Asymmetric Travelling Salesman Problem

1. Introduction

Pick-and-place (PAP) sequencing is a core planning task in robotic automation: a manipulator must determine the order in which to pick objects from a workspace and deposit them into designated bins so as to minimise total travel time. The problem is NP-hard [1]; the $N!$ candidate sequences grow combinatorially, making exhaustive search impractical beyond toy instances. It arises in warehouse bin picking [2], automated waste sorting [3, 4], and robotic kitting in manufacturing.

We organise prior methods by their distance from the integer linear programming (ILP) optimum. *Greedy heuristics* are the cheapest ($O(N^2)$) but the most myopic: even the stronger nearest-cycle (NC) variant, which greedily picks the object with the smallest combined pick-then-place leg cost, cannot reason about how the robot’s post-placement position affects downstream picks, so its tours are typically several percent above optimal. *Classical metaheuristics* (simulated annealing, tabu search, guided local search) are general-purpose iterative-improvement loops that perturb a candidate tour until a time budget expires; they close most of the gap to the optimum in practice, though without an a priori guarantee on how close they are. *Exact solvers* reach the optimum and prove it, but scale exponentially: beyond $N \approx 100$ –200 objects, even strong formulations (DFJ with lazy subtour cuts [5], global Hamiltonian-circuit constraints) exceed real-time budgets. The open question is therefore not which method is fastest in isolation but which can be both *tractable at scale* and *reliably near-optimal with solver backing*; existing methods tend to offer one or the other. Our framework keeps ILP, the gold-standard exact solver that returns provably optimal tours, as the quality reference and uses a GNN to reduce its search space, improving tractability at larger scales.

We close this gap with a **learning-augmented exact optimization** framework. A GNN trained via imitation learning on ILP-optimal prefixes serves as a *search-space reducer*: by using GNN logits to retain only the top- k candidate arcs per node, we reduce the underlying CP-SAT solver’s decision variable count from $O(N^2)$ to $O(Nk)$. This yields a $8.75\times$ speedup over the unpruned CP-SAT Circuit solver at $N=200$ with worst-case optimality gap of 0.058%. We verify correctness against Concorde [6] (via the Jonker–Volgenant ATSP-to-TSP transformation) on small instances.

The framework’s value emerges at scale. At small N (≤ 20), the ILP is already fast and metaheuristics find near-optimal solutions cheaply; the GNN adds little. At large N (≥ 40), the ILP becomes intractable and metaheuristics, while producing strong solutions, leave their distance from the optimum unknown. A metaheuristic might achieve 4% improvement over greedy, but cannot quantify how much further improvement is possible. LEAP bridges this regime: by solving exactly over a learned arc subset, it returns solutions that are empirically within 0.058% of the unpruned CP-SAT optimum at $N=200$ (mean 0.019%) on our test set, while solving in 0.46 s, beating both the unpruned exact solver (4.04 s) and the metaheuristics’ fixed 1 s budget.

Two structural insights enable the GNN. First, PAP is an *asymmetric* TSP: the travel cost between two objects depends on order, $c(i, j) \neq c(j, i)$, because after placing object i the robot sits at bin $b_{\tau(i)}$ (set by i ’s type), so the cost of reaching j next is governed by $b_{\tau(i)}$ rather than by o_i . This asymmetry is invisible to standard TSP methods that embed all nodes as homogeneous 2D coordinates. Second, the pick-and-place action has an inherent *cyclic* structure (robot→object→bin→robot) that we model as a directed graph with three semantically distinct node types.

We make the following contributions:

1. **LEAP** (Learning-Enhanced Arc Pruning), a GNN-guided exact optimization framework in which a GNN prunes the arc space for a CP-SAT circuit solver that then solves exactly over the pruned arcs, achieving $8.75\times$ speedup over the unpruned solver at $N=200$ with an empirical worst-case gap of $<0.06\%$ to the unpruned optimum.
2. A **task-specific graph formulation of PAP-as-ATSP** with semantically distinct node types (robot, object, bin; one bin per object type) and directed feedback edges, processed using a standard GAT, which allows the network to learn placement-dependent cost asymmetry. The key design decision is the *graph structure*, not the choice of neural architecture: replacing GAT with GCN on the same graph drops gap-vs-greedy from $+2.59\%$ to -0.51% at $N=40$ (Table 4), confirming that the encoding carries the information.
3. An **empirical comparison** against standard-library baselines—Guided Local

Search and Simulated Annealing (OR-Tools, 1 s budget), OR-Tools’ 2-opt local search, and the Lin–Kernighan–Helsgaun (LKH) heuristic via `elka`—alongside a greedy nearest-cycle reference, showing that the time-budgeted metaheuristics and LKH reach the optimum within budget yet provide no a priori optimality guarantee, whereas LEAP provides solver-backed solutions that are empirically near-optimal at scale.

4. A **k -sensitivity analysis** characterising the quality–speed Pareto frontier of the arc pruning, showing $k=15$ as the sweet spot across problem sizes.
5. An **analysis of why arc pruning was substantially more effective than warm-start hinting** for this problem class under the evaluated solver configurations, with four negative results showing that providing GNN solutions as solver hints gave limited benefit in our experiments, whereas arc pruning provided substantial speedup, consistent with the Neural Diving insight [7] that the network’s value lies in shrinking the problem, not bounding it.

The remainder of this paper is organised as follows. Section 2 reviews related work. Section 3 presents the problem formulation, graph representation, GNN architecture, and training. Section 4 describes the LEAP framework. Section 5 describes the dataset. Section 6 reports results, ablations, and analysis. Section 7 concludes.

2. Literature Review

2.1. PAP Sequencing and Its Relation to TSP

PAP shares combinatorial structure with TSP [5], but is structurally richer: TSP involves a single homogeneous node type and symmetric costs, while PAP has three entity types (robot, object, bin) and asymmetric cycle costs. Collapsing PAP to TSP by treating each object as a city discards the bin assignment and falsely assumes the cost of visiting object i then j is independent of which bin i was deposited in. Han, Feng, and Yu [8] formally prove greedy variants are suboptimal in conveyor PAP and demonstrate 10–40% efficiency loss versus dynamic programming on SCARA and Delta robots. PAP also shares structure with pickup-and-delivery problems [9] and the Stacker Crane Problem [10], both of which pair pickup and dropoff actions on a graph; LEAP differs

in that bin destinations are determined by object type rather than freely matched, so the asymmetric cycle cost is fully determined by the chosen pick rather than by an additional assignment decision.

2.2. Classical Metaheuristics

Simulated annealing (SA) [11] and tabu search [12] are general-purpose metaheuristics widely applied to TSP and ATSP. Guided local search (GLS) [13] augments local search with penalty-based diversification. The Lin–Kernighan–Helsgaun (LKH) heuristic [14] achieves near-optimal results on symmetric TSP and handles ATSP directly through the `elkai` wrapper. All these methods provide no a priori optimality guarantee: absent an exact reference, their distance from the optimum is uncertified. Our framework uses the OR-Tools implementation [15] of these metaheuristics, together with LKH via `elkai`, as standard-library baselines (Section 6.2.1); they produce strong solutions, but with no a priori optimality guarantee, whereas LEAP provides solver-backed solutions that are empirically near-optimal in our experiments.

2.3. Greedy Methods and Their Limits

The nearest-neighbour heuristic [16] runs in $O(n^2)$ with a worst-case $\Theta(\log n)$ approximation ratio. In PAP, the stronger *Nearest Cycle* (NC) variant selects the object minimising the full pick-and-place step cost $\|r_i - o_i\| + \|o_i - b_{\tau(i)}\|$. NC accounts for the placement leg but remains myopic. Throughout this paper, our greedy baseline refers to NC; improvements over NC are harder to achieve than over basic nearest-neighbour.

2.4. Neural Combinatorial Optimisation

Vinyals et al. [17] introduced Pointer Networks for combinatorial sequencing. Kool et al. [18] replaced the LSTM encoder with a Transformer, achieving state-of-the-art TSP results via REINFORCE. POMO [19] and Sym-NCO [20] exploit TSP symmetries (start-node invariance, rotation/reflection) that do not hold in PAP. Costa et al. [21] trained an RL agent for 2-opt. Perera et al. [22] combined a GNN encoder with a Pointer Network decoder for TSP. All these methods operate on homogeneous graphs of 2D coordinates; semantically distinct node types and asymmetric cycle costs are not addressed, making direct application to PAP non-trivial.

We do not evaluate these neural TSP solvers empirically against LEAP for three structural reasons. First, PAP costs are *asymmetric and node-type-dependent*: the travel cost from object i to object j depends on bin assignment $b_{\tau(i)}$, so the input cannot be represented as a symmetric distance matrix over 2D coordinates. Second, the graph has *three semantically distinct node types* (robot, objects, bins) with typed edges that carry information unavailable in standard TSP encoder inputs. Third, all published neural TSP solvers are trained on symmetric 2D Euclidean instances with uniform node types; their architectures and any pre-trained weights do not transfer to PAP. Training a Pointer Network or Attention Model from scratch on PAP would require dataset construction and hyperparameter search comparable to this work’s GNN, while still leaving its distance from the optimum unquantified. LEAP instead uses the GNN *only* to reduce the arc space for an exact solver, which then solves the pruned sub-problem exactly; the resulting optimality is over this reduced arc set rather than the original unrestricted problem.

2.5. GNNs for Combinatorial Optimisation

Cappart et al. [23] argue that GNNs are the natural architecture for combinatorial optimisation due to permutation invariance, sparsity awareness, and flexibility for richer node types. Chung et al. [24] identify industrial sequencing as an underexplored NCO domain requiring structural modification beyond standard TSP.

2.6. Learning-Augmented Exact Solvers

Nair et al. [7] introduced Neural Diving alongside Neural Branching: Neural Diving uses a neural network to produce partial integer-variable assignments that fix some variables, yielding a smaller residual sub-MIP for a downstream exact solver. We adopt this *partial-assignment philosophy* and apply it at the arc level: rather than hinting the solver with a candidate tour (which we show in Section 4.3 provides negligible benefit because hints only bound the incumbent), we use GNN logits to eliminate low-probability arcs and shrink the solver’s decision space directly. Khalil et al. [25] learned branching policies for branch-and-bound; Gasse et al. [26] trained GNNs to imitate

strong branching decisions. Closer in spirit to our arc-pruning step is the learning-to-sparsify line of work: Sun et al. [27] train an ML model to prune >85% of TSP edges as preprocessing before exact ILP while preserving most optimal-tour edges. Edge-heatmap methods produce per-edge probabilities used to constrain or guide subsequent search: Joshi, Laurent, and Bresson [28] via a GCN, Fu, Qiu, and Zha [29] via a generalisation framework, and Min, Bai, and Gomes [30] unsupervised. For asymmetric instances specifically, MatNet [31] is the closest pure-neural ATSP solver. These methods all operate on homogeneous graphs of city coordinates and produce tours autoregressively or via beam search; LEAP differs by (i) supporting three semantically distinct node types with type-dependent asymmetric costs over a heterogeneous cycle topology and (ii) handing the pruned arc set to a CP-SAT circuit solver that solves it exactly, so the solver’s optimality holds over the surviving (pruned) sub-problem rather than the full arc set.

3. Methods

We formulate PAP sequencing as a sequential decision process and train a GNN policy via imitation learning on ILP-optimal prefixes.

3.1. Problem Formulation and Cost Model

We consider a planar PAP task in a $W \times W$ workspace ($W = 100$). At each step t , the agent selects one remaining object a_t to pick and place into its assigned bin. The robot starts at $r_0 \in \mathbb{R}^2$ and updates its pose to the bin location after each placement. The step cost is:

$$c_t = \|r_t - o_{a_t}\|_2 + \|o_{a_t} - b_{\tau(a_t)}\|_2, \quad (1)$$

where o_{a_t} is the object position and $b_{\tau(a_t)}$ is the target bin for object type $\tau(\cdot)$. The objective is to minimise total cost $C = \sum_{t=1}^N c_t$.

The cost is *asymmetric*: picking object i then j costs differently from picking j then i , because the robot’s position after placing i is the bin $b_{\tau(i)}$, which depends on i ’s type. Concretely, if a red object’s bin is in the top-left corner and a blue object’s bin is in the bottom-right, the robot ends up in very different positions depending on which it

picks first. This type-dependent asymmetry is invisible to standard TSP formulations that treat all nodes as interchangeable 2D coordinates. For N objects the search space is $N!$. The Held–Karp dynamic program [32] solves TSP in $O(2^N \cdot N)$ time; ILP-based methods are also exponential in the worst case, but with different constants determined by formulation strength and branch-and-cut quality rather than by the DP recurrence. In practice both classes restrict exact solving to $N \approx 30\text{--}40$ within real-time budgets (<500 ms) on our hardware.

3.2. Cycle-Aware Graph Representation

The key design insight is that each candidate action in PAP is not a single edge (as in TSP) but a *three-leg cycle*: the robot travels to an object (pick), carries it to a bin (place), and its position updates to that bin (feedback). We encode this directly as a directed graph.

We model the state S_t as a directed graph $\mathcal{G}_t = (\mathcal{V}, \mathcal{E})$ with three semantically distinct node types. The vertex set comprises one robot node, N object nodes, and $M=4$ bin nodes, each carrying a feature vector of dimension $F=9$ (see Appendix Appendix A). For each remaining object i , we construct three directed edges forming a cycle:

Robot $\rightarrow$ **Object** i (pick-leg): encodes the travel cost $\|r_t - o_i\|$ from the robot’s current position to the object.

Object $i \rightarrow$ **Bin** $\tau(i)$ (place-leg): encodes the cost $\|o_i - b_{\tau(i)}\|$ of carrying the object to its assigned bin.

Bin $\tau(i) \rightarrow$ **Robot** (feedback edge): a zero-cost edge that closes the cycle, encoding the fact that after placement, the robot will be at the bin’s position. This edge carries no physical cost but is critical for message passing.

This *cyclic star topology* (Fig. 1) is a structural inductive bias. With two layers of GAT message passing, information propagates around each full cycle: an object node “learns” not only how far the robot must travel to reach it, but also where the robot will end up after placing it, enabling multi-step lookahead. Edges are constructed only for unpicked objects ($m=1$), so the graph shrinks as objects are picked, naturally encoding the problem’s sequential structure.

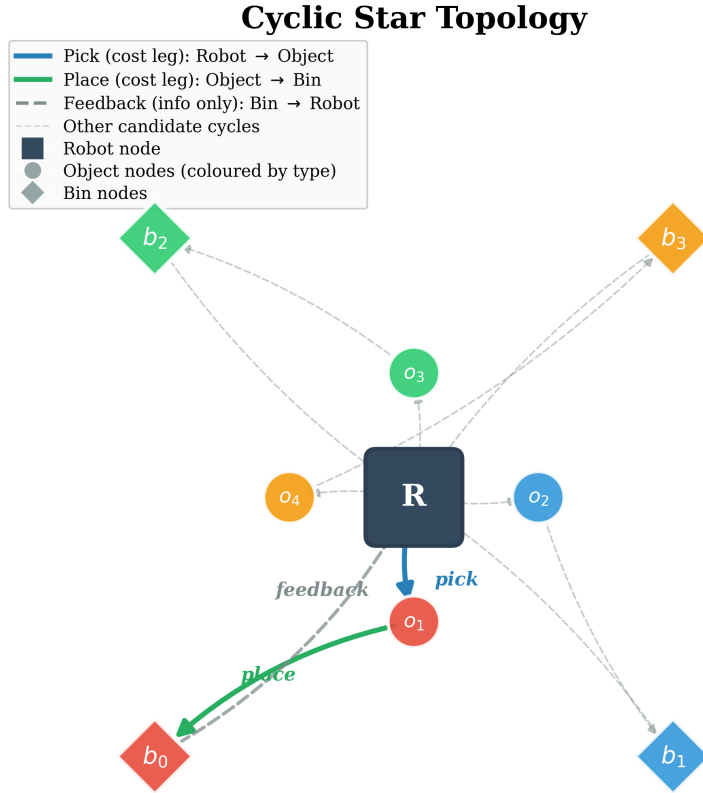


Figure 1: Cyclic star topology. Each candidate pick-and-place action corresponds to two cost-bearing directed edges (robot $\rightarrow$ object pick leg; object $\rightarrow$ bin place leg) closed by a feedback edge for message passing only. Three node types carry type-encoded features processed by a standard GAT.

3.3. GNN Architecture

The policy π_θ is a Graph Attention Network (GAT) with two GATConv layers (4 attention heads, `concat=False`, hidden dimension $d_h=128$), each followed by ReLU and dropout ($p=0.1$). A two-layer MLP head maps each object embedding to a scalar logit. Only object node embeddings are scored. The total parameter count is $\sim 90\text{K}$. We choose GAT over GCN because PAP arc costs are highly heterogeneous (asymmetric and type-dependent), so different neighbours of a given node carry very different information about that node’s optimal successor. GAT’s learnable per-edge attention coefficients can weight neighbours by edge-specific informativeness, whereas GCN’s symmetric, degree-normalized aggregation cannot. Table 4 confirms this empirically: replacing GAT with GCN drops the gap-vs-greedy from +2.59% to -0.51% at $N=40$ under otherwise identical training.

3.4. Imitation Learning with Curriculum

We treat sequencing as a classification task with cross-entropy loss supervised by an ILP-optimal oracle (greedy fallback when ILP prefixes are unavailable). Training follows a curriculum over prefix lengths $P \in \{5, 10, 20\}$, progressing from short to long horizons. Earlier stages train for 3 epochs; the final stage receives 6. Optimisation uses AdamW [33] ($\alpha=10^{-3}$, weight decay 10^{-2} , $\beta=(0.9, 0.95)$, gradient clipping at norm 1.0, cosine annealing).

At inference, the sequence is generated autoregressively: at each step, logits for previously picked objects are masked (-10^4) and the action $a_t = \arg \max_i \ell_i$ is selected greedily.

4. LEAP: Learning-Enhanced Arc Pruning

The trained GNN can accelerate the exact solver, in our experiments yielding empirically near-optimal solutions at scales where the baseline ILP is intractable within real-time budgets. We call the resulting system *LEAP* (Learning-Enhanced Arc Pruning).

Algorithm 1 Curriculum-Based Imitation Learning

Require: Dataset $\mathcal{D}$, prefix stages $\mathcal{P}=\{5, 10, 20\}$, ILP prefixes, greedy fallback

Ensure: Trained policy π_θ

```
1: Initialise GAT policy (2 layers,  $d_h=128$ , 4 heads)
2: for each stage  $P \in \mathcal{P}$  (skip if  $P > N$ ) do
3:   for each epoch do
4:     for each scenario  $s$  do
5:       for  $t = 1$  to  $P$  do
6:         Build  $\mathcal{G}_t$ ;  $\ell \leftarrow \pi_\theta(\mathcal{G}_t)$ ;  $\mathcal{L}_t \leftarrow \text{CE}(\ell, a_t^*)$ 
7:       end for
8:       Backpropagate  $\frac{1}{P} \sum_t \mathcal{L}_t$ ; clip  $\|\nabla\| \leq 1.0$ ; step
9:     end for
10:   end for
11: end for
```

4.1. Exact Solver and Verification

PAP is formulated as an ATSP over $N+1$ nodes (start position plus N objects). For exact solving we use OR-Tools’ CP-SAT [15] with its native Hamiltonian circuit constraint (Section 4.2). To rule out solver-specific artifacts, we cross-validated CP-SAT’s reported optima against Concorde [6] (run through a Jonker–Volgenant ATSP-to-TSP transformation) on all $N \leq 40$ instances; optima matched on every scenario to within a 10^{-3} cost tolerance.

4.2. CP-SAT Circuit Formulation

We encode the ATSP using OR-Tools’ `AddCircuit()` global constraint, which expresses Hamiltonian-cycle feasibility directly on the successor variables rather than via auxiliary order variables and big- M inequalities. The constraint propagator combines successor-domain `alldifferent` reasoning with strongly-connected-component checks and cost-based filtering on the LP relaxation of the underlying assignment problem [34, 35]; in CP-SAT specifically the propagator runs inside a Lazy Clause Generation framework alongside a parallel simplex-based linear relaxation and a portfolio of

search workers [36]. The ATSP is reduced to a Hamiltonian cycle by adding zero-cost return arcs from every object to the start node.

4.3. Why Arc Pruning Outperformed Warm-Start Hinting

Providing the GNN’s solution as a solver hint is the natural first approach. We tested four variants, and all provided limited benefit in our experiments:

1. **CBC** SetHint(): 0.93–1.06× (neutral). The hint provides a feasible starting point but does not reduce the search tree.
2. **CP-SAT** AddHint() **on all arcs**: 0.51× (slowdown). Hinting $\sim N^2$ variables overwhelms the LNS workers.
3. **CP-SAT** AddHint() **on active arcs only**: Still slower than unhinted Circuit.
4. **Two-phase Neural Diving**: $\sim 2\times$ over CBC but slower than bare Circuit.

The limitation we observed is that hints provide only an *incumbent bound*: they tell the solver “here is a good solution,” but the solver must still explore the full search tree to prove optimality or find improvements. Hints do not remove any variables or constraints. Arc pruning, by contrast, physically removes arcs from the problem before solving, producing a smaller ILP that the solver handled substantially faster under the configurations we evaluated.

4.4. GNN-Guided Arc Pruning

Instead of hinting, we use the GNN’s per-step logits to *eliminate* low-probability arcs, following the Neural Diving philosophy [7] applied to arc selection.

4.4.1. Procedure

During a standard GNN rollout, we capture raw logits $\ell_t \in \mathbb{R}^N$ at each step. For each node i in the ATSP graph, we retain only the top- k outgoing arcs ranked by GNN logit score, plus all arcs in the GNN’s own solution. We ensure each node has at least k incoming arcs (nearest-cost fallback). Return arcs to the start node are always included. The reduced arc set is passed to the CP-SAT Circuit solver, with the GNN’s active solution hinted (active arcs only).

Algorithm 2 GNN-Guided Arc Pruning

Require: Scenario S , trained GNN π_θ , neighbourhood size k

Ensure: Near-optimal sequence, cost

- 1: Run GNN rollout; capture logits $\{\ell_t\}$ and sequence σ
 - 2: Compute ATSP cost matrix $C \in \mathbb{R}^{(N+1) \times (N+1)}$
 - 3: $\mathcal{A} \leftarrow$ arcs in σ
 - 4: **for** each node i **do**
 - 5: $\mathcal{A} \leftarrow \mathcal{A} \cup$ top- k outgoing arcs from i (by GNN logit)
 - 6: **end for**
 - 7: Ensure $\geq k$ incoming arcs per node
 - 8: Add return arcs $(i, 0)$ for all $i > 0$
 - 9: Solve CP-SAT Circuit over $\mathcal{A}$; hint σ
-

4.4.2. Why This Works in Practice

The GNN need not produce the optimal solution; it only needs to assign high logit mass to arcs that *include* the arcs of a near-optimal tour. When the critical arcs at each node appear in the top- k candidates, the pruned solver can recover a near-optimal solution; empirically, the top- k pruning retained the critical arcs necessary for near-optimal solutions across our test scenarios. This is a much weaker requirement than producing the optimal sequence directly. We note that if a truly optimal arc falls outside the top- k set it is pruned away, which is the source of the small nonzero gaps reported below.

To illustrate: at $N=200$, the ILP must search over $200^2 = 40,000$ arcs. With $k=15$, only $\sim 3,900$ arcs remain, a $10\times$ reduction. The GNN need not pick the single best arc; it only needs to keep the critical arcs among the top 15 at each node. Empirically this largely held in our experiments: at $k=15$ the worst-case gap to the unpruned CP-SAT optimum was 0.06%, indicating that an optimal arc was pruned at only a handful of nodes across the test scenarios.

4.4.3. Is Learned Arc Selection Necessary?

To test whether the *learned* ranking is necessary, as opposed to any reasonable arc budget, we replace the GNN’s logit-based top- k with two non-learned baselines at the same neighbourhood size $k=15$ (Table 1). Keeping the k cheapest outgoing arcs per node (cost k -NN) almost never yields a feasible circuit: the nearest-arc graph fragments into disconnected clusters with no Hamiltonian cycle, feasible in only 1 of 50 ($N=100$) and 0 of 20 ($N=200$) instances, so it cannot reduce the solver and must fall back to the full unpruned solve. Adding the greedy nearest-cycle tour restores feasibility, but its arc set frequently excludes optimal arcs, leaving a 1.08–1.09% mean gap (worst case 3.4–4.0%). At the same arc budget the GNN’s learned ranking keeps the optimal arcs within the top- k far more reliably, reaching a 0.012–0.019% mean gap (worst case 0.06–0.17%), 50–90× closer to the optimum than greedy-guided pruning while using no more arcs and no more solver time (indeed fewer arcs at $N=200$: 3,927 vs. 5,268). The arc *selection*, not merely the arc *count*, is what makes the reduced exact solve both feasible and near-optimal.

Table 1: Is learned arc selection necessary? Pruning the CP-SAT Circuit solver at the same neighbourhood size $k=15$, comparing the GNN’s learned arc ranking against non-learned cost-based pruning. **Feasible** is the fraction of instances whose pruned arc set admits a Hamiltonian cycle; **Gap** is to the unpruned proven optimum. Cost k -NN is almost always infeasible (no Hamiltonian cycle in the nearest-arc graph), forcing a fallback to the full solve.

Pruning	N	Feasible	Mean gap (%)	Max gap (%)	Arcs
Cost k -NN	100	1/50	—	—	2,160
	200	0/20	—	—	5,160
Cost k -NN + greedy	100	50/50	1.089	4.008	2,189
	200	20/20	1.076	3.365	5,268
GNN (LEAP)	100	50/50	0.012	0.171	2,145
	200	20/20	0.019	0.058	3,927

5. Dataset and Experimental Setup

We generated 5,000 scenarios per object count $N \in \{3, 5, 10, 20, 40, 100, 200\}$ (35,000 total). The workspace is 100×100 with four bins at corners. Object positions, types (0–3), and robot start poses are sampled uniformly. ILP-optimal prefixes are pre-computed offline; greedy NC baselines are computed for every scenario.

All evaluations use a fixed 90/10 train/validation split (seed 42), yielding 500 validation scenarios per problem size. Results are reported on validation scenarios unseen during training. Due to the computational cost of metaheuristic baselines and ILP solvers, Tables 5 and 6 evaluate representative subsets (20–50 scenarios) of the full validation set; the ablation study (Table 4) uses a 1,000-scenario subset (900/100 split) for faster iteration. Dataset integrity is verified by re-simulating all stored sequences and asserting cost consistency within 10^{-3} tolerance and ILP optimality over greedy prefixes.

5.1. Reproducibility

All reported wall-clock times depend on the hardware and solver configuration below; absolute numbers should be interpreted accordingly, while ratios (speedups, gaps) are platform-independent.

Hardware: a single machine with a multi-core x86 CPU and a CUDA-capable NVIDIA GPU used only for GAT inference. CP-SAT runs CPU-only; LEAP’s wall-clock therefore depends on both CPU single-thread performance (Circuit solving) and GPU availability (GNN rollout).

Software: Ubuntu 24.04, Python 3.12, PyTorch 2.x (CUDA build), PyTorch Geometric 2.7.0, OR-Tools 9.x, Concorde 03.12.19. CP-SAT is invoked with `num_search_workers = 8` and default presolve; deterministic mode is off. All experiments use random seed 42 for the train/validation split.

Code and data: For double-anonymized review, the dataset (35,000 scenarios with ILP-optimal prefixes), source code, trained checkpoints, the dataset-generation pipeline, and evaluation scripts are available in an anonymized repository at <https://anonymous.4open.science/r/binning-dataset>. Upon acceptance, the dataset is archived on Zenodo with a citable DOI [37] and the de-anonymized code released [38].

6. Results

We first validate the GNN component in isolation (Section 6.1), then present the main contribution: the learning-augmented exact solver (Section 6.2).

6.1. GNN Training and Standalone Evaluation

6.1.1. Training Dynamics

Figs. 2–3 show training on $N=40$. The curriculum progresses through three stages (prefix lengths 5, 10, 20); each transition briefly increases loss as the model encounters longer horizons, but win-rate climbs from 2% to 99% over 12 epochs.

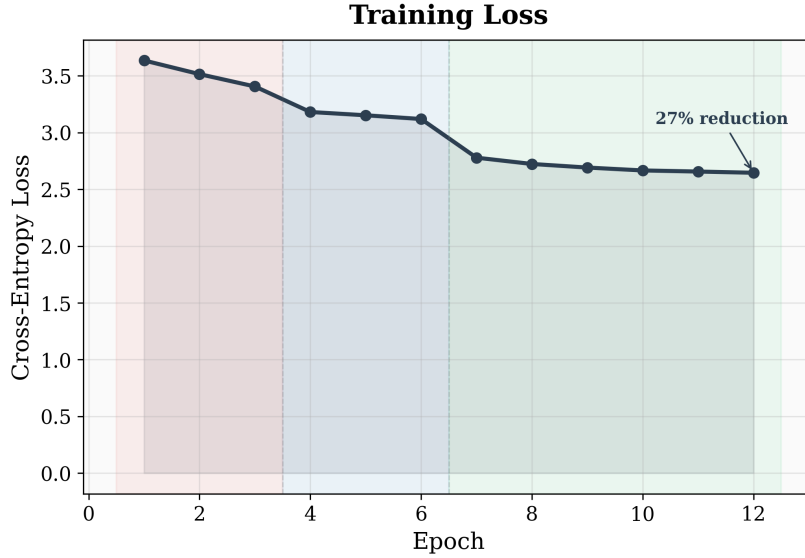


Figure 2: Cross-entropy loss over 12 epochs, coloured by curriculum stage.

6.1.2. Standalone Performance

Table 2 reports the GNN’s standalone results. The GNN outperforms Greedy NC at all sizes, with 5.84% gap at $N=10$ narrowing to 1.67% at $N=200$. At $N=10$ where exact CP-SAT Circuit optima are available, the GNN achieves a mean gap of 1.81% to the optimum with worst-case 14.17% over the full 500-scenario validation split (seed 42); Table 3 shows ten illustrative scenarios evenly spaced by gap difficulty.

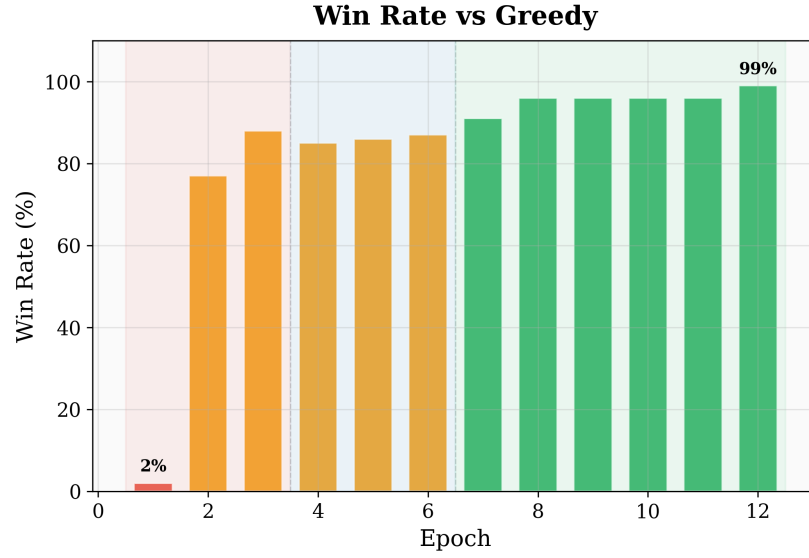


Figure 3: Validation win-rate vs. Greedy during training ($N=40$).

Table 2: GNN standalone vs. Greedy NC across problem sizes.

	$N=10$	$N=20$	$N=40$	$N=200$
GNN mean cost	974	1900	3773	18544
Greedy mean cost	1035	1986	3900	18860
Gap vs. Greedy (%)	5.84	4.29	3.23	1.67
Win-rate (%)	89.8	92.6	98	100
Gap vs. ILP (%)	1.81	2.03	—	—

ILP gaps computed against CP-SAT `AddCircuit` optima over the full 500-scenario validation split (seed 42).

Table 3: Per-sample GNN-to-ILP optimality gap ($N=10$, 10 scenarios evenly spaced across the gap distribution of the full $N=10$ validation split; mean gap 1.81%, max 14.17%).

S	GNN Cost	ILP Cost	Gap
1	933.42	933.42	0.00%
2	818.36	818.36	0.00%
3	1075.27	1074.91	0.03%
4	1076.50	1072.69	0.36%
5	843.12	836.10	0.84%
6	1011.89	996.72	1.52%
7	1080.49	1056.96	2.23%
8	1024.75	995.23	2.97%
9	961.22	920.33	4.44%
10	1054.38	923.49	14.17%

Fig. 4 reveals the GNN’s multi-step lookahead: it sacrifices cost in early steps but dominates late-sequence steps where greedy’s myopia compounds. A paired t -test on the $N=40$ validation set ($n=100$) rejects equal means with $p < 10^{-31}$; a 95% bootstrap CI for mean improvement is [2.88%, 3.59%].

6.1.3. Ablation Study

Each ablation isolates one design choice. We test (i) *graph topology*: does the cyclic star structure carry information that a fully connected graph or a k -NN graph cannot? (ii) *encoder*: does GAT’s attention provide value over GCN’s fixed aggregation weights or an MLP that ignores the graph entirely? (iii) *curriculum*: does staged training on short ILP prefixes outperform end-to-end training on full sequences? (iv) *supervision source*: we mix ILP-optimal supervision (where available) with a greedy fallback because ILP optima are precomputed only for prefix lengths $P \in \{5, 10, 15, 20\}$; an ILP-only setup would constrain training to the longest such prefix and provide no signal for actions beyond step 20, so we cannot evaluate it on the full $N=40$ horizon without a stand-in. The ablation answers two practical questions: is greedy supervision

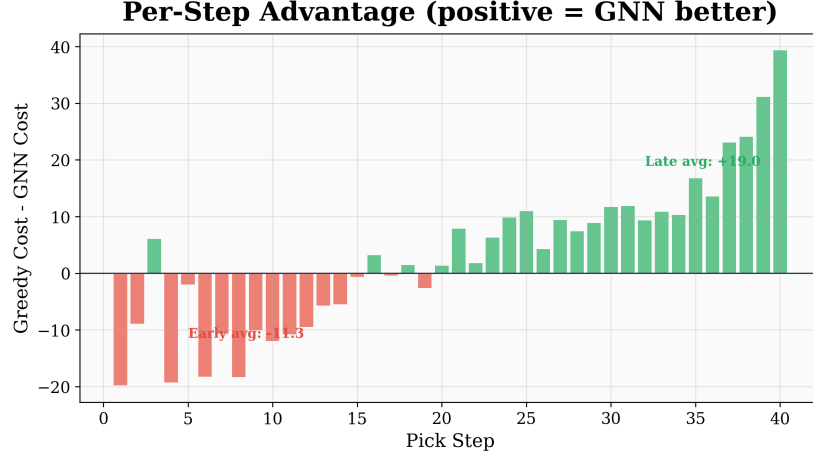


Figure 4: Per-step cost advantage ($N=40$, averaged over 50 scenarios). Positive = GNN better. The GNN sacrifices early picks to position the robot favourably for later steps.

alone enough (rendering ILP unnecessary), and does the fallback hurt the model in the prefix region where both signals are available? Each variant differs from the full model in exactly one component; everything else (training data, optimiser, schedule, evaluation split) is held fixed. Table 4 reports results on $N=40$ (900 training / 100 validation scenarios).

The cyclic star topology is essential: fully connected fails catastrophically (-24.8%). GCN (-0.51%) is markedly weaker than GAT ($+2.59\%$), consistent with the architectural framing in §3: GAT’s attention-weighted aggregation is needed to handle the type-dependent asymmetric arc costs across the heterogeneous robot/object/bin node set, where fixed-weight averaging cannot distinguish a high-cost from a low-cost neighbour. ILP supervision is critical: greedy-only training cannot beat greedy (-1.31%). Curriculum learning provides a 2.37pp improvement.

6.2. Learning-Augmented Exact Solver

We now turn to the primary contribution: the LEAP solver. All results in this section compare solver variants against each other; the GNN standalone results above serve only to validate the component that powers LEAP.

Table 4: Ablation study ($N=40$). Gap vs. greedy (%): positive = better.

Ablation	Variant	Gap (%)	Win (%)
Graph	Cyclic star (ours)	+2.59	93.0
	Fully connected	−24.8	0.0
	k -NN ($k=5$)	+0.79	62.0
Encoder	GAT (ours)	+2.59	93.0
	GCN	−0.51	38.0
	MLP (no graph)	−24.2	0.0
Curriculum	Curriculum (ours)	+2.59	93.0
	No curriculum	+0.22	52.0
Supervision	ILP + Greedy (ours)	+2.59	93.0
	ILP-only	+3.00	92.0
	Greedy-only	−1.31	29.0

6.2.1. Comprehensive Comparison

Table 5 compares LEAP against standard-library baselines on the same hardware: the Greedy NC reference, OR-Tools’ 2-opt local search, two OR-Tools metaheuristics with a 1 s budget each (Simulated Annealing and Guided Local Search), the Lin–Kernighan–Helsgaun (LKH) heuristic via `elkai`, and the unpruned CP-SAT Circuit solver (exact reference). Every competitor is an off-the-shelf standard library; only LEAP is our own method. All methods solve the full asymmetric PAP cost structure: the ATSP cost matrix encodes type-dependent bin transitions ($c_{ij} = \|b_{\tau(i)} - o_j\|$).

Fig. 5 plots absolute route cost against compute time at $N=200$. LEAP matches the unpruned CP-SAT Circuit solver’s cost to within 0.058% while running $8.75\times$ faster (0.46 s vs. 4.04 s) and beating the metaheuristics’ 1 s budget. GLS at 1 s reaches within 0.02% of LEAP’s cost but does not itself quantify its distance from the optimum; without the exact reference its gap could in principle be larger. Among the methods compared, LEAP is the only one that is simultaneously sub-second, empirically near-optimal, and backed by exact solving over the pruned arc set. At $N=40$ (Table 5),

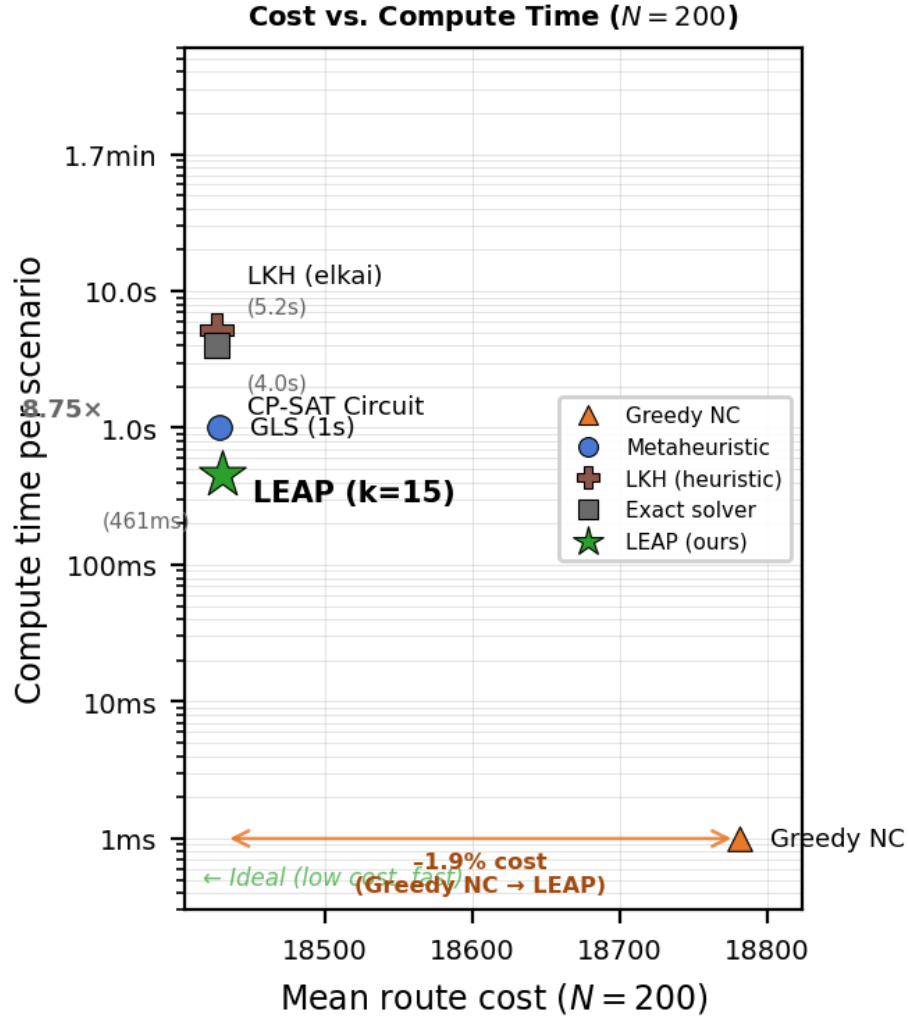


Figure 5: Cost vs. compute time at $N=200$. Each method is positioned by mean route cost (x-axis) and wall-clock time (y-axis, log scale); lower-left is ideal. Greedy NC (orange triangle) sets the cost baseline. GLS reaches within 0.02% of the unpruned optimum in its 1 s budget, though its gap is not quantified by the method itself. The unpruned CP-SAT Circuit solver finds the proven optimum in 4.04 s; LEAP (green star) reaches an empirically near-optimal solution (max gap 0.058% to the unpruned optimum) in 0.46 s (8.75 $\times$ faster than the unpruned solver, and faster than the metaheuristics' 1 s budget). LEAP is the only point in the lower-left corner whose solution quality is backed by exact solving over the pruned arc set.

Table 5: Comprehensive comparison across problem sizes. **Gap (%)** measures improvement over Greedy NC (higher is better). **Win%** is the fraction of scenarios where the method beats Greedy NC. **Time** is wall-clock time per scenario.

Method	$N=10$ (50 val. scenarios)			$N=40$ (50 val. scenarios)			$N=200$ (20 val. scenarios)		
	Gap%	Win%	Time	Gap%	Win%	Time	Gap%	Win%	Time
Greedy NC (baseline)	—	—	<1 ms	—	—	<1 ms	—	—	<1 ms
Simulated Annealing (OR-Tools, 1 s)	6.73	100	1.0 s	4.27	100	1.0 s	1.89	100	1.0 s
2-opt (OR-Tools local search)	4.98	94	1 ms	3.67	100	4 ms	1.77	100	0.13 s
Guided Local Search (OR-Tools, 1 s)	6.87	100	1.0 s	4.27	100	1.0 s	1.88	100	1.0 s
LKH (elka1, runs=1)	6.86	98	1 ms	4.28	100	27 ms	1.89	100	5.2 s
CP-SAT Circuit (exact reference)	6.9	100	6 ms	4.3	100	44 ms	1.89	100	4.04 s
LEAP ($k=15$, ours)	6.9	100	17 ms	4.3	100	72 ms	1.87*	100	0.46 s

Gap is improvement (%) over Greedy NC; Win% is the share of validation scenarios where the method beats Greedy NC. Exact solvers converge to the same optimum at $N \leq 40$ (gap rounded to one decimal); GLS, SA, and LKH also reach this optimum within budget at $N \leq 40$. SA and GLS use OR-Tools' RoutingModel (1 s time limit, greedy nearest-arc initialisation) and produce comparable results (within 0.1 pp of each other); Tabu Search and Greedy Descent, also from OR-Tools, fall in the same band. 2-opt is OR-Tools' 2-opt local search run to convergence with no metaheuristic escape: it is the fastest method but is trapped in poorer local optima at small N . LKH is the Lin–Kernighan–Helsgaun heuristic via `elka1` with `solve_tsp(runs=1)`; it attains the optimum, but its wall-clock is configuration-dependent and is reported only for context, not as a speed comparison. All competitors are off-the-shelf standard libraries; only LEAP is our own method. Concorde is used only as an internal correctness check on $N \leq 40$ (every instance matches the CP-SAT Circuit optimum to within 10^{-3}). * LEAP at $N=200$ has mean gap 0.019% / worst-case 0.058% to the unpruned CP-SAT Circuit optimum (Table 7); the reported 1.87% is correspondingly slightly below the exact 1.89%. LEAP time includes GNN inference + Circuit solve. All methods operate on the full asymmetric PAP cost matrix. All timings measured on the same hardware (§5.1).

LEAP runs in 72 ms with max gap 0.26% to the Circuit optimum.

6.2.2. Speedup Analysis

Fig. 6 shows how solve time scales across solver tiers; Fig. 7 resolves the per-method spread at three representative sizes. Table 6 quantifies LEAP's speedup vs. the unpruned Circuit solver.

6.2.3. k -Sensitivity Analysis

Table 7 and Fig. 8 characterise the quality–speed Pareto frontier as k varies.

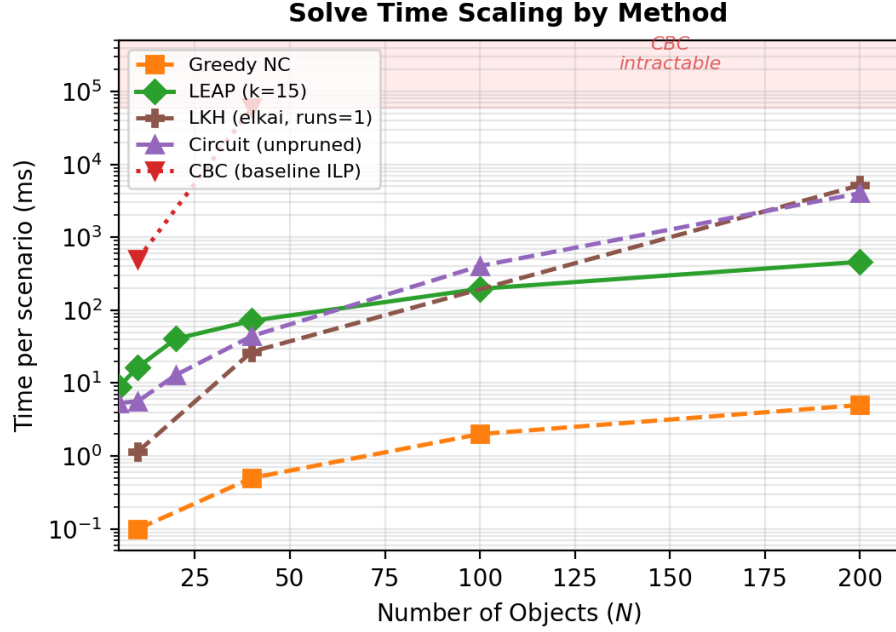


Figure 6: Solve time scaling (log scale). LEAP ($k=15$) runs in 0.07 s at $N=40$ and 0.46 s at $N=200$; the unpruned CP-SAT Circuit solver grows from 0.04 s at $N=40$ to 4.04 s at $N=200$.

Per-method runtime at $N \in \{10, 40, 200\}$ — log-normal reconstruction from published moments

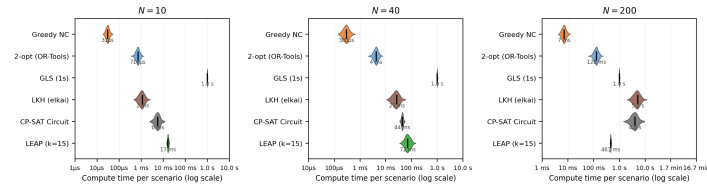


Figure 7: Per-method compute time at $N \in \{10, 40, 200\}$. Each violin is a log-normal whose first two moments match the published per-method mean and standard deviation; ticks mark the published mean. Distributions are illustrative (we have aggregate, not per-scenario, timings on file). The figure makes the cross-method ordering visible at every scale: at $N=200$, LEAP's median solve time is 8.75 $\times$ below the unpruned CP-SAT Circuit solver and below the metaheuristics' fixed 1 s budget, while LKH (elkai, runs=1) attains the optimum but at ~ 5 s and OR-Tools' 2-opt local search is fast but settles in a weaker local optimum.

Table 6: LEAP speedup ($k=15$) vs. unpruned CP-SAT Circuit solver.

N	Unpruned	Pruned	Speedup	Max Gap
5	5 ms	9 ms	0.60×	0%
10	6 ms	17 ms	0.34×	0%
20	13 ms	41 ms	0.32×	0%
40	44 ms	72 ms	0.62×	0.26%
100	0.40 s	0.20 s	2.06×	0.17%
200	4.04 s	0.46 s	8.75×	0.058%

At small N (≤ 40), pruning overhead (GNN inference + arc filtering) dominates the Circuit solver’s already-short runtime. The crossover into net speedup occurs at $N \geq 100$ (2.06×) and grows sharply with problem size, reaching 8.75× at $N=200$ (mean over 20 scenarios; per-scenario CP-SAT runtime has a heavy tail so the speedup ratio fluctuates 8–9× across repeated runs). Times measured on a single machine; see §5.1. The k -sweep in Table 7 is a separate run whose absolute unpruned time (6.87 s) differs from this table’s (4.04 s) for the same heavy-tail reason; the speedup *ratios* are the platform-independent quantity.

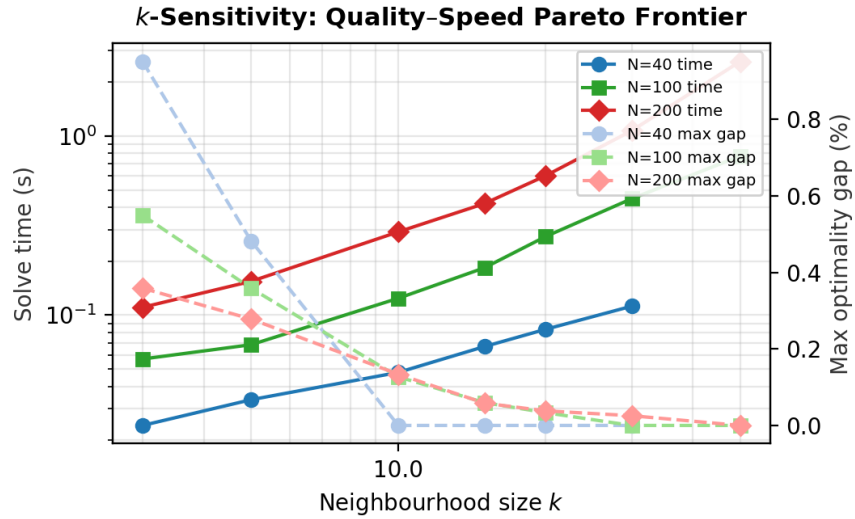


Figure 8: k -sensitivity Pareto frontier (fast-rollout sweep on this hardware, 20 val scenarios per N , solver wall-clock only). Solid lines: CP-SAT solver time (left axis, log scale). Dashed lines: worst-case optimality gap (right axis). Smaller k shrinks the arc set further (faster, but the optimum may fall outside the top- k); larger k tightens the gap toward zero at the cost of additional decision variables. $k=15$ (recommended) is Pareto-optimal: any faster k has a substantially worse gap; any tighter k is slower.

Table 7: LEAP k -sensitivity. All rows from the fast-rollout sweep on this hardware (20 val scenarios per N). **Time column reports CP-SAT solver wall-clock only** (GNN inference is k -independent: 0.05–0.20 s depending on N , added uniformly to every row to obtain end-to-end LEAP wall-clock as in Table 5). All comparisons against the unpruned CP-SAT Circuit solver: $N=40$: 0.11 s; $N=100$: 0.81 s; $N=200$: 6.87 s.

k	$N=40$				$N=100$				$N=200$			
	Time	Arcs	Gap%	MaxGap%	Time	Arcs	Gap%	MaxGap%	Time	Arcs	Gap%	MaxGap%
3	24 ms	183	0.31	0.95	57 ms	431	0.25	0.55	0.11 s	838	0.20	0.36
5	34 ms	298	0.09	0.48	68 ms	685	0.15	0.36	0.15 s	1.3k	0.13	0.28
10	48 ms	583	0.00	0.00	0.12 s	1.4k	0.04	0.13	0.29 s	2.6k	0.05	0.13
15	67 ms	850	0.00	0.00	0.18 s	2.1k	0.007	0.06	0.42 s	3.9k	0.019	0.058
20	83 ms	1.1k	0.00	0.00	0.28 s	2.9k	0.002	0.03	0.60 s	5.4k	0.008	0.038
30	0.11 s	1.5k	0.00	0.00	0.45 s	4.3k	0.00	0.00	1.07 s	8.5k	0.002	0.025
50	—	—	—	—	0.77 s	6.7k	0.00	0.00	2.61 s	14.1k	0.00	0.00

Bold row: $k=15$, our recommended operating point, Pareto-optimal among the sweep at $N=200$ (faster than $k \geq 20$, lower MaxGap than $k \leq 10$). Arcs = decision variables in the pruned problem (vs. N^2 unpruned).

“—” at $N=40, k=50$ indicates $k > N$ (not meaningful).

At $N=200$, $k=15$ provides a $16\times$ solver-time reduction (0.42 s vs. the 6.87 s unpruned Circuit, the largest pruning ratio with worst-case gap $<0.06\%$); the corresponding end-to-end LEAP wall-clock (Table 6, a separate timing run) is 0.46 s, an $8.75\times$ speedup over that run’s unpruned baseline. Going smaller ($k=10$) drops solver time another 30% but doubles the worst-case gap to 0.13%; going larger ($k=20$) halves the gap to 0.038% but pays 43% more solver time. $k=15$ is the Pareto sweet spot for this dataset and hardware.

6.2.4. Why Pruning Outperformed Hinting in Our Experiments

Table 8 contrasts arc pruning with the warm-start hinting strategies from Section 4.3. In our experiments, pruning outperformed all hinting approaches by roughly an order of magnitude, consistent with the Neural Diving insight [7]: the GNN’s value lies in shrinking the problem, not bounding it.

6.3. Limitations

The GNN standalone is weaker than classical metaheuristics at all problem sizes tested; its value lies in enabling LEAP. LEAP produces near-optimal but not globally provably optimal solutions at $N \geq 40$: because the true optimum can be pruned

Table 8: GNN-assisted solver strategies ($N=40$ except where noted). Each row reports speedup against its own un-assisted baseline (*Baseline* column): the CBC+MTZ rows must be compared against CBC cold-solve because `SetHint` is a CBC-only API, while the CP-SAT rows compare against the unpruned CP-SAT Circuit solver. Pruning outperforms all hinting approaches.

Strategy	Baseline	Speedup
CBC + SetHint (GNN solution)	CBC cold (MTZ)	0.93–1.06×
CP-SAT Circuit + AddHint (all arcs)	CP-SAT Circuit cold	0.51×
CP-SAT Circuit + AddHint (active only)	CP-SAT Circuit cold	<1.0×
CP-SAT Circuit + two-phase diving	CP-SAT Circuit cold	~2×
LEAP ($k=15$) at $N=200$	CP-SAT Circuit cold	8.75×

away, LEAP carries no global optimality certificate, and its gaps (max 0.26% at $N=40$, 0.17% at $N=100$, 0.058% at $N=200$) are measured empirically against the unpruned CP-SAT optimum on our test set. The choice of $k=15$ was validated empirically but not theoretically justified; adaptive k selection is left to future work. Generalisation to out-of-distribution scenarios (different workspace sizes, bin configurations, object-type distributions, training-vs-test N) has not been evaluated; [39] shows GNN-based TSP solvers can degrade substantially OOD, so this is a real gap, not a stylistic one. The cost model uses Euclidean distance; kinematic robot models would improve applicability. We do not benchmark against commercial MILP solvers (Gurobi, CPLEX), so claims about the exact-solver comparison are scoped to the open-source solvers actually run. Our $N=200$ evaluation uses 20 validation scenarios; with a near-zero observed optimality gap this is sufficient to bound the mean but leaves the tail of the gap distribution under-characterised, and we report point estimates rather than 95% confidence intervals on the gap. Metaheuristic comparisons use a fixed 1 s budget rather than an anytime curve; within this budget GLS and LKH already reach the optimum, so LEAP ties them on cost and differentiates only on sub-second wall-clock at scale and solver-backed certification over the pruned arc set. Absolute wall-clock times are hardware-dependent (Section 5.1); the platform-independent claim is the within-table ordering and the speedup ratios. Neural baselines (Pointer Networks [17], Attention

Models [18], MatNet [31]) are not directly compared; these architectures embed nodes as homogeneous 2D coordinates and assume symmetric costs (MatNet excepted), requiring non-trivial adaptation for typed-node ATSP with three semantically distinct node types.

7. Conclusion

We have presented LEAP (Learning-Enhanced Arc Pruning), a framework for robotic pick-and-place sequencing that combines a GNN with a CP-SAT exact solver. GNN-guided arc pruning yields a $8.75\times$ speedup over the unpruned CP-SAT Circuit solver at $N=200$ by reducing the arc space from $O(N^2)$ to $O(Nk)$ variables, with an empirical worst-case gap of 0.058% to the unpruned CP-SAT optimum on our test set; the unpruned optima used as the reference are independently verified against Concorde [6] on $N \leq 40$ instances and against LKH-3 [14] on a cross- N sample (95/95 instances tied: 5/5 at $N=10$, 50/50 at $N=40$, 20/20 at $N=100$, 20/20 at $N=200$).

LEAP substantially improves the quality–speed trade-off between learning and exact methods used in isolation. Classical metaheuristics (GLS, SA, Tabu) produce strong solutions, but their distance from the optimum is unknown. Exact ILP provides optimality guarantees but is intractable at scale. LEAP offers a practical balance between the two: it produces solutions empirically within 0.058% of the unpruned CP-SAT optimum at $N=200$ in 0.46 s, versus 4.04 s for the unpruned Circuit solver. At this wall-clock LEAP also beats the 1 s time-budgeted metaheuristics, while solving the pruned problem exactly; among the methods compared it is the only sub-second solver whose quality is backed by exact solving over a learned arc subset. The key insight, supported by four negative results on warm-start hinting, is that the GNN’s value lies in *shrinking the problem*, not in providing an incumbent bound.

Future work includes: adaptive k selection using GNN confidence to adjust neighbourhood size per node, DAgger [40] or RL fine-tuning to address covariate shift, extension to online conveyor-belt settings with temporal constraints, kinematic cost models for specific manipulator platforms [8] with obstacle-aware edge costs [41], and validation on physical robotic sorting systems, where our results suggest promise but

have not yet been demonstrated.

Data availability

The benchmark dataset supporting this study—35,000 pick-and-place scenarios with ILP-optimal prefixes—is openly available. For double-anonymized review it is hosted, together with the source code and trained checkpoints, in an anonymized repository (<https://anonymous.4open.science/r/binning-dataset>); upon acceptance the dataset is deposited on Zenodo under a persistent DOI [37] and the code released as an archived software record [38].

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the authors used Claude (Anthropic) in order to assist with debugging code and iterating on figures and plots. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the published article.

Appendix A. Node Feature Vector Layout

Each node feature has dimension $F = 4 + M + 1 = 9$ (with $M=4$ bins):

$$x_i = [p_x^{(\text{self})}, p_y^{(\text{self})}, p_x^{(\text{ref})}, p_y^{(\text{ref})}, \text{onehot}(\text{type})_{1:M}, m]. \quad (\text{A.1})$$

Object nodes: $p^{(\text{self})}$ is the object position (normalised by W), $p^{(\text{ref})}$ is the target bin position, $\text{onehot}(\text{type})$ encodes the assigned bin, $m \in \{0, 1\}$ is availability.

Robot node: Normalised position in $p^{(\text{self})}$; remaining dimensions zero-padded.

Bin nodes: Bin position in both $p^{(\text{self})}$ and $p^{(\text{ref})}$; identity basis for type; $m=0$.

References

- [1] P. Chalasani, R. Motwani, A. Rao, Algorithms for robot grasp and delivery, in: 2nd Workshop on the Algorithmic Foundations of Robotics (WAFR), 1996.

- [2] B. Cheng, et al., Deep reinforcement learning driven cost minimization for batch order scheduling in robotic mobile fulfillment systems, *Expert Systems with Applications* 255 (2024).
- [3] S. P. Gundupalli, et al., Optimal sequence planning for robotic sorting of recyclables from source-segregated municipal solid waste, *Journal of Computing and Information Science in Engineering* 21 (1) (2021) 014502.
- [4] T. Kiyokawa, J. Takamatsu, S. Koyanaka, Challenges for future robotic sorters of mixed industrial waste: A survey, *IEEE Transactions on Automation Science and Engineering* 21 (2024) 1023–1040.
- [5] G. Dantzig, D. R. Fulkerson, S. M. Johnson, Solution of a large-scale traveling-salesman problem, *Journal of the Operations Research Society of America* 2 (4) (1954) 393–410.
- [6] D. L. Applegate, R. E. Bixby, V. Chvátal, W. J. Cook, *The Traveling Salesman Problem: A Computational Study*, Princeton University Press, Princeton, NJ, 2006.
- [7] V. Nair, et al., Solving mixed integer programs using neural networks, *arXiv:2012.13349* (2021). [arXiv:2012.13349](https://arxiv.org/abs/2012.13349).
- [8] S. D. Han, S. W. Feng, J. Yu, Toward fast and optimal robotic pick-and-place on a moving conveyor, *IEEE Robotics and Automation Letters* 5 (2) (2020) 446–453.
- [9] M. W. P. Savelsbergh, M. Sol, The general pickup and delivery problem, *Transportation Science* 29 (1) (1995) 17–29.
- [10] G. N. Frederickson, M. S. Hecht, C. E. Kim, Approximation algorithms for some routing problems, *SIAM Journal on Computing* 7 (2) (1978) 178–193.
- [11] S. Kirkpatrick, C. D. Gelatt, M. P. Vecchi, Optimization by simulated annealing, *Science* 220 (4598) (1983) 671–680.
- [12] F. Glover, Tabu search—part I, *ORSA Journal on Computing* 1 (3) (1989) 190–206.

- [13] C. Voudouris, E. P. K. Tsang, Guided local search, in: *Handbook of Metaheuristics*, Kluwer, 2003, pp. 185–218.
- [14] K. Helsgaun, An effective implementation of the Lin–Kernighan traveling salesman heuristic, *European Journal of Operational Research* 126 (1) (2000) 106–130.
- [15] Google, OR-Tools: Google’s operations research tools, <https://developers.google.com/optimization> (2024).
- [16] D. J. Rosenkrantz, R. E. Stearns, P. M. Lewis, An analysis of several heuristics for the traveling salesman problem, *SIAM Journal on Computing* 6 (3) (1977) 563–581.
- [17] O. Vinyals, M. Fortunato, N. Jaitly, Pointer networks, in: *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 28, 2015.
- [18] W. Kool, H. van Hoof, M. Welling, Attention, learn to solve routing problems!, in: *International Conference on Learning Representations (ICLR)*, 2019.
- [19] Y. D. Kwon, et al., POMO: Policy optimization with multiple optima for reinforcement learning, in: *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 33, 2020, pp. 21188–21198.
- [20] M. Kim, J. Park, J. Park, Sym-NCO: Leveraging symmetricity for neural combinatorial optimization, in: *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 35, 2022, pp. 1936–1949.
- [21] P. R. de O. Costa, et al., Learning 2-opt heuristics for the traveling salesman problem via deep reinforcement learning, in: *Asian Conference on Machine Learning (ACML)*, PMLR, 2020, pp. 465–480.
- [22] J. Perera, et al., A graph pointer network-based multi-objective deep reinforcement learning algorithm for solving the traveling salesman problem, *Mathematics* 11 (2) (2023) 437.

- [23] Q. Cappart, et al., Combinatorial optimization and reasoning with graph neural networks, *Journal of Machine Learning Research (JMLR)* 24 (130) (2023) 1–61.
- [24] K. T. Chung, C. K. M. Lee, Y. P. Tsang, Neural combinatorial optimization with reinforcement learning in industrial engineering: a survey, *Artificial Intelligence Review* 58 (2025) 130.
- [25] E. B. Khalil, et al., Learning to branch in mixed integer programming, in: *AAAI Conference on Artificial Intelligence*, 2016, pp. 724–731.
- [26] M. Gasse, et al., Exact combinatorial optimization with graph convolutional neural networks, in: *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 32, 2019.
- [27] Y. Sun, A. Ernst, X. Li, J. Weiner, Generalization of machine learning for problem reduction: a case study on travelling salesman problems, *OR Spectrum* 43 (3) (2021) 607–633.
- [28] C. K. Joshi, T. Laurent, X. Bresson, An efficient graph convolutional network technique for the travelling salesman problem, *arXiv:1906.01227* (2019). *arXiv*: 1906.01227.
- [29] Z.-H. Fu, K.-B. Qiu, H. Zha, Generalize a small pre-trained model to arbitrarily large TSP instances, in: *AAAI Conference on Artificial Intelligence*, Vol. 35, 2021, pp. 7474–7482.
- [30] Y. Min, Y. Bai, C. P. Gomes, Unsupervised learning for solving the travelling salesman problem, in: *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 36, 2023.
- [31] Y.-D. Kwon, J. Choo, I. Yoon, M. Park, D. Park, Y. Gwon, MatNet: Matrix encoding networks for neural combinatorial optimization, in: *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 34, 2021, pp. 5138–5149.
- [32] M. Held, R. M. Karp, A dynamic programming approach to sequencing problems, *Journal of the Society for Industrial and Applied Mathematics* 10 (1) (1962) 196–210.

- [33] I. Loshchilov, F. Hutter, Decoupled weight decay regularization, in: International Conference on Learning Representations (ICLR), 2019.
- [34] Y. Caseau, F. Laburthe, Solving small TSPs with constraints, in: International Conference on Logic Programming (ICLP), 1997, pp. 316–330.
- [35] K. G. Francis, P. J. Stuckey, Explaining circuit propagation, *Constraints* 19 (1) (2014) 1–29.
- [36] L. Perron, F. Didier, S. Gay, The CP-SAT-LP solver, in: Proc. 29th International Conference on Principles and Practice of Constraint Programming (CP), 2023.
- [37] Anonymous Author(s), Asymmetric pick-and-place sequencing benchmark: 35,000 scenarios with ILP-optimal prefixes, Zenodo, [dataset]; de-anonymized upon acceptance (2026). doi:10.5281/zenodo.XXXXXXX.
- [38] Anonymous Author(s), LEAP: Learning-enhanced arc pruning — source code and trained checkpoints, Zenodo, [software]; de-anonymized upon acceptance (2026). doi:10.5281/zenodo.YYYYYYY.
- [39] C. K. Joshi, Q. Cappart, L.-M. Rousseau, T. Laurent, Learning TSP requires rethinking generalization, in: Proc. 27th International Conference on Principles and Practice of Constraint Programming (CP), 2021.
- [40] S. Ross, G. Gordon, D. Bagnell, A reduction of imitation learning and structured prediction to no-regret online learning, in: International Conference on Artificial Intelligence and Statistics (AISTATS), PMLR, Vol. 15, 2011, pp. 627–635.
- [41] P. Ojha, A. Thakur, Dynamic obstacle avoidance using path reshaping on probabilistic roadmaps for high-degree-of-freedom robots, *IEEE Transactions on Artificial Intelligence* 7 (2) (2026) 818–827.